

Introduction to Operating Systems

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Me (Dr. Sam Ogden)

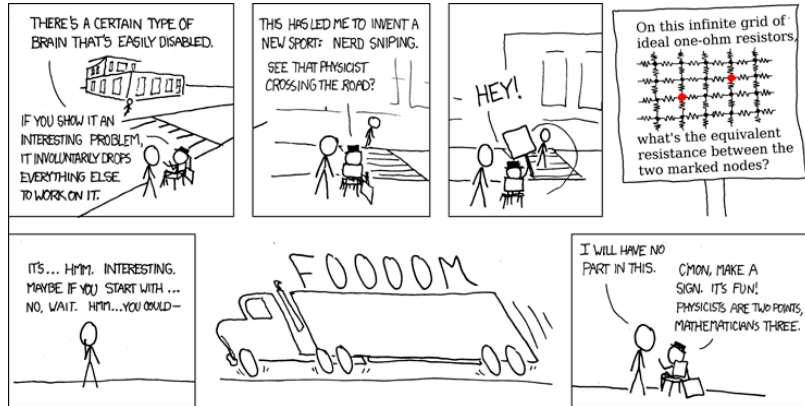
- Office hours:
- Mondays 8-9am
- Saturdays 9-10am
- By Appointment

- BS (Math&EE), Univ. of Vermont
- MS (CS), Univ. of Vermont
- PhD (Comp. Sci.), WPI

- Rock climbing
- Running
- Reading
- Getting lost (“exploring”)
- Ice cream
- Good bread
- Nerdy questions

- Product Engineer @ IBM/GF (5 years)
- Grad Student @ WPI (5 years)
- Professor @ CSUMB (4 years)

Nerd Sniping



Course TAs



Ryan Hopper

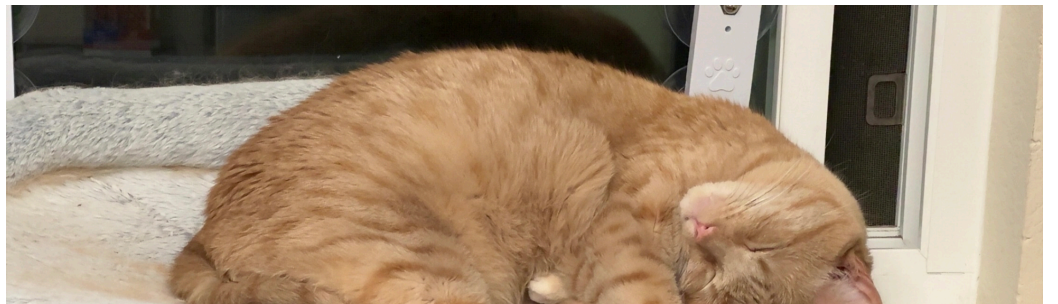


Daniel Solano

Getting Help

- Use the course site and discussion channels first
- Ask questions early rather than batching them until the deadline
- Include screenshots, commands, and error text when reporting a problem

Cat Tax



A quick caveat about my schedule

Next week (e.g. the 26th of June to the 6th of July) I'll have limited internet access.

- Slack should work, but zoom may be spotty

I'll try to keep office hours but might need to cancel and have to keep up on slack

Course Information

Course Overview

Class Structure

- Most days the class will try to go like this:
- lecture (50 minutes)
- break (10 minutes)
- lecture (35 minutes)
- lab (15 minutes)

Textbook: OSTEP

Operating Systems: Three Easy Pieces

- aka **OSTEP**
- Great book (with one caveat)
- Pretty readable for a textbook
- Completely free!



Operating Systems: Three Easy Pieces

Grading and Assignments

Grading

- Three big groups of grades:
 - (35%) Exams
 - (30%) Participation
 - (35%) Programming Assignments
- You must get at least a 40% in each of these to pass

Grading

Bucket	Value
Programming Assignments	35%
Exams and Quizzes	30%
Interaction	35%
Kayla's Surveys	(1%)

Exams

- Tests (35%):
 - four exams
 - 8.75% each
 - closed book, but 1 single-sided handwritten page of notes
 - generally cumulative — about half the material will be from previous sections (but percentage varies)
 - Practice Quizzes are provided beforehand!
 - (and are worth bonus points!)

Participation

- Class Attendance (10%):
 - many classes start with in-class reading quizzes
 - reading is linked on weekly schedule
 - More likely to have quizzes when class looks empty
- Learning logs (5%)
 - Reflect on what you learned this week, or what you struggled with
 - The guidelines are not bullet points but meant as an algorithm
- Lab activities (5%):
 - due Sunday night

Programming Assignments

OS is not a programming class – it's a concept class

PAs are meant to give context to what we are learning

- Assignments are in the class github repository
- You should only change `student_code.c` and `student_code.h`
 - All other files submitted are discarded
- 5-6 assignments throughout the class
 - each assignment is 1 week long

- They are in the C programming language
 - C is meant to be very simple – often confusingly so
 - Very similar to Java and C++
 - Drawing out memory contents can often help a lot!

Programming Assignment (continued)

- Functionality is tested with unit tests (via the `make` command)

- Scoring is broken up per function/goal (e.g. `string__at` or `end-to-end`)
 - Not all are worth the same, so aim for high-value first

- Some tests are “RESERVED” so you only get to see the names and

Make sure you complete the first 2 labs before starting the homework

- They set up **docker** which we use in class
- They also walk you through the assignment structure

Programming Assignment: Critical Info

- The most important thing to do is to **make sure your code compiles**
 - The starter code will compile
 - If your code doesn't compile then you can't get any points at all
- The autograder runs **2x a day** (at noon and midnight)
 - It will determine your grade, or tell you if something went wrong

Weekly Study Notes: help your future self

These study notes are for you to go back and use as a basis for studying

How to approach these notes:

1. identify the topics that were covered
2. explain ideas in your own words
3. identifying least-understood topics
4. noting “aha” moments
5. explaining the nature of the confusion
6. asking questions
7. wondering about ideas, connections, applications
8. deeper reflection

How to get help

TA Office Hours!



How to get help: what's okay

- Asking your instructor!
 - Seriously, come by office hours
- Asking your TAs!
 - They've taken the class and are nice!
- Asking LLMs for clarification or guidance!
 - “What does limited direct execution mean?”
 - “Can you make me some practice problems for process scheduling?”

But don't trust LLMs entirely!

How to get help: what's not okay

- Getting `[fill in the blank]` to write code for you
 - Friends
 - Enemies (for numerous reasons this is a bad idea)
 - LLMs
- The goal is to understand the problems and learn how to approach them!
 - Any code you turn in you are responsible for!

Dealing with Challenges in Class

Take a breath

- I gather Operating Systems has a reputation as a scary class
 - It is a lot of information
 - But it's meant to be about simple steps
- Remember:
 - The core idea of OS is **simplicity**
 - We (the TAs and me) are here to help
 - If you're struggling, ask!

Mental Health

- Mental health is super important
- If you're feeling stressed or anxious about an assignment then take a break on it for a bit!
 - (This is the real reason I want you to look at assignments early — so you have time to walk away for a bit)
- If you're feeling stuck then ask me or the TAs!

How to solve hard problems

Trying to solve a hard problem in one sitting is a bad idea.

How to solve hard problems (cont'd)

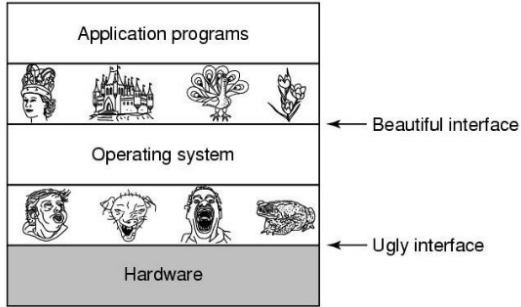
- Good diffuse-mode activators:
 - take a walk, take a shower, go to the gym, sleep, meditate, listen to music
- Less effective:
 - play video games, surf the web, talk to friends, text friends, TV

Some things to remember

- Focus on growth and mastery
- Start homework early
- Drop by my office to chat
- Please call me just Sam (or Dr./Prof. Ogden)

Operating Systems

What's an Operating System?

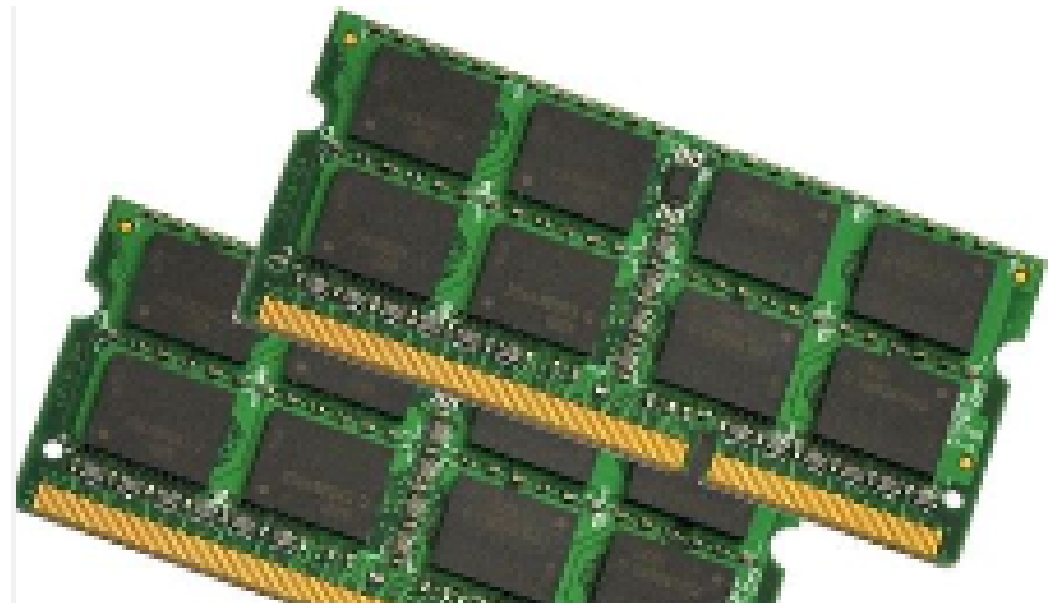


Tanenbaum

OS is a “systems” course

- In a CS systems course, we think about:
- physical resources shared by multiple people or applications
- What are the resources of a computer system?

- CPU
- Disk
- Memory (RAM)
- I/O



OS is a “systems” course

- How is this different from other programming classes?
 - We need to consider **how programs interact**
 - We our porgram is **no longer running in isolation**
 - In fact, we know it isn't!
 - We have **concrete limits** to resources!

The Core Ideas of OS

Problem Statement

Core Question:

How to provide resources to users in an easy-to-use, reliable, fair, efficient, and secure way?

- Operating Systems is an amazing subject because this problem is everywhere:

- restaurants
- gyms
- colleges
- roads
- ...

Solution Approach for Systems

The approach for systems problems:

1. Begin with the minimal solution that solves the problem
2. Identify bottlenecks and add complexity only where necessary
3. Repeat step 2 until requirements are satisfied

The Core Idea of Systems (and OS)

Operating systems are designed to work and to be simple.

The most important thing to remember is we want to find the simplest solution that works well enough

An [imperfect] Example: cake

- Resource: A cake
 - | ■ For now, let's assume it's a round cake
- Users: 7 users total
- Challenge:
 - | ■ How do we divide up the cake?

Core ideas of OS

- Our discussions often have two parts
 - **Mechanism:** What low-level support is needed to make this happen
 - e.g. a knife for the cake example
 - **Policy:** How do we use the mechanism to achieve particular goals?
 - e.g. ensure everybody has an equal sized piece of cake

Example: Processor as a shared resource

- Virtual resource: each app *thinks* it has its own CPU
- Questions
 - Which app should get to use the physical CPU?
 - If OS lets an app have the CPU, how does it get control back?

Example Guiding questions

- What are the virtual resources or services offered to users? Are they easy to use?
- How to ensure fair sharing of resources between users?
- How to protect users from each other, and protect the system from users?
- What workloads do we use to measure performance?
- What metrics do we use to measure performance?

- How efficiently are resources managed?

Recap

- Welcome to OS!
- Three main chunks of assignments
 - | ■ Programming Assignments are graded 2x a day
- Take time with this class
 - | ■ If you leave everything to Tuesday night you're gonna have a bad time
- When we approach problems, let's think about how we can do thing *simply*
 - | ■ And *good enough* is enough